

## Paths

Creating a path is simple, and has only three basic steps, you begin the path, you draw a number of lines, or curves, and you stroke or fill the path. The simplest path that you can draw is a line. All that needs is two parameters, the x and y ordinates of the points to which you want to draw the line:

```
beginPath(); // begin a new path
lineTo(x, y); // add a line to the specified location
stroke(); // stroke the path
```

You may wonder, where does this line start! To which the answer is, whatever the final location after the previous operation. If you have a specific location in mind, you can use the `moveTo` method. Here is how you would draw a line from (10, 15) to (25, 45):

```
ctx.beginPath();
ctx.moveTo(10, 15);
ctx.lineTo(25, 45);
```

If now you wanted to draw a line from this point to (50, 90), all you would need to do is another `lineTo` call, no need for `moveTo` again since it will continue from there.

In these examples, until you call `stroke`, nothing will be displayed on screen. It is important to note that `stroke` will only outline your path, if you want to fill the shape you have created, you can use the `fill` method instead. If you use the `fill` method, it will automatically close an open shape. If you are building a complex shape, and in the end just want to connect the starting and ending points, you can use `closePath()`.

We glossed over `moveTo` before but it is actually quite important. It is a way to move the pointer on the canvas without drawing anything – like lifting your pen. You can use it to create complex disconnected shapes that are part of one path.

## Curved paths

Straight paths can only take you so far. You will probably need to create curves, or there is little point to a drawing surface. Luckily the canvas has some powerful methods for creating all kinds of curves. There are essentially three kinds of curves that you can create on the canvas, each more complex than the last.

The first is the simple arc, or part of a circle. The simplest way to draw one is using the `arc` method: `arc(x, y, radius, startingAngle, endingAngle, anticlockwise)`

The x, and y specify the centre of the circle as expected, while the starting and ending angle specify what part of the circle to draw, and the last parameter specifies whether the arc is to be drawn clockwise or anticlockwise. The angles here are in radians, not degrees, so to draw an entire circle you can set the starting

# \*pointers

>> Nuggets of cool code at work



## \*QuakeCon 2012 Panel - Virtual Insanity

>> John Carmack (id Software), Michael Abrash (Valve), and Palmer Luckey (Oculus).

Duration > 1:02:17

<http://dvw.in/QSh4Fc>



## \*Google Drive SDK: write your first Drive app in Python

>> During this Google Developer session, you'll learn how to build command line Python apps for Google Drive.

Duration > 23:41

<http://dvw.in/NVC1LE>



## \*A view of Windows Phone 8

>> Here's a look at the initial impressions of Microsoft's new Windows Phone 8 operating system.

Duration > 09:47

<http://dvw.in/OtIsaE>



## \*The Google ADK and Android

>> Ben Heck checks out the Google ADK, a device that connects your Android phone to an Arduino MCU and plans a project with it.

Duration > 15:28

<http://dvw.in/PORX61>

angle to 0, and the ending angle to  $2\pi$  or in JavaScript `Math.PI * 2`.

Other than `arc` we have `quadraticCurveTo` and `bezierCurveTo`. There is also `arcTo` which is a little confusing unless you know exactly how it works.

While the `arc` method is simple enough to explain here—and so we have—the other methods aren't that simple. All we can say is that if you have used an application like Photoshop or Illustrator you will find those concepts easier to understand. They allow you to create more complex curves than just circles. The point is, if you are looking to create curves in the canvas, you have some research to do.

## Strokes and Fills

Till now our strokes and fills have been dull and black. You will be pleased to know that you do have a lot of control over these things.

For setting the width of the stroke you have `lineWidth`, while for setting the stroke style you have `strokeStyle`. For setting the fill style you have `fillStyle`. Both `strokeStyle` and `fillStyle` work in similar ways. You can provide it a colour just like you would in CSS:

```
fillStyle = "red";
fillStyle = "#FFFF00"; // yellow
fillStyle = "rgb(128, 0, 128)"; // purple
fillStyle = "rgba(255, 255, 255, 0.4)"; // 40% transparent white
```

It is also possible to use linear or radial gradients with strokes and fills, however that is beyond the scope of this article. Remember to set these values before you call the `fill` or `stroke` method.

## Conclusion

As you can see the canvas has a basic but functional and powerful API for manipulating content that isn't restricted by HTML rules and hierarchy. Everything you do on the canvas is in your control.

Read that again, "everything you do on canvas is under your control", this is also a disadvantage. You cannot easily create clickable buttons or controls on canvas, nor are they accessible or tabable. All that has to be done by you. Canvas is best for multimedia content and animations. There is still a lot about the Canvas API that you should learn about, such as drawing images on the canvas, using transforms, accessing the canvas data etc. Of course there is the whole 3D aspect of the canvas that requires some knowledge of WebGL. Consider this your starting point. >

Read the original article online on devworx. Visit [www.devworx.in](http://www.devworx.in)